



Time Series Clustering for Enhanced Dynamic Allocation in A/B Testing

Emmanuelle Claeys, Myriam Maumy-Bertrand, Pierre Gançarski

► To cite this version:

Emmanuelle Claeys, Myriam Maumy-Bertrand, Pierre Gançarski. Time Series Clustering for Enhanced Dynamic Allocation in A/B Testing. ECML PKDD 2024, Sep 2024, Vilnius, Lithuania. hal-04612889

HAL Id: hal-04612889

<https://hal.science/hal-04612889v1>

Submitted on 14 Jun 2024

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.

Time Series Clustering for Enhanced Dynamic Allocation in A/B Testing

Emmanuelle Claeys¹ (✉), Myriam Maumy-Bertrand², and Pierre Gançarski³

¹ IRIT, University of Toulouse III, 31400, Toulouse France
`emmanuelle.claeys@irit.fr`

² University of Technology of Troyes, 10430 Rosières-près-Troyes, France

³ ICUBE, University of Strasbourg, 67411 Illkirch-Graffenstaden, France

Abstract. An A/B-Test is a method for evaluating online experiments on target items and observing which A/B/C/... variations are better through log reports and statistical analysis of the rewards earned by each variation. Recent advancements in A/B-Tests through reinforcement learning encompass dynamic allocation employing multiarmed bandits (MAB). MABs provides A/B-Tests with fast identification of the best variation (A or B) and helps limit the loss of the test i.e. the cost of exploring low-reward variation. When partial information is available before assigning variations, dynamic allocation is extended to the contextual multiarmed bandit problem (CMAB). Current state-of-the-art approaches for empirically estimating the context-dependent reward function for each variation demonstrate strong performance in limiting test loss and personalized tests. However, few studies have addressed this problem in the context of variable-sized time series. This paper presents a new reinforcement learning methodology to handle A/B-Tests with variable-sized time series as context information. We provide two new methods that obtain a minimization of the cumulative regret with a soft computational cost. This paper also provides numerical results on real A/B-Test datasets, in addition to public data, to demonstrate an improvement over traditional methods.

Keywords: A/B-TEST · Mutliarmed bandit · Time series.

1 Introduction

In many domains, experimental evaluation is necessary for assessing the relevance of modifications made to an existing entity according to one or more objectives. For instance, an e-marketing team can look for the best modification of a web page design to increase sales [6]. The original variant (A) and its variations (B/C/...) are compared in parallel in a real environment. This leads to the exploration-exploitation dilemma which opposes the cost of learning the best variation (exploration phase) and the benefit obtained by using it in the future (exploitation phase). To tackle this dilemma, novel A/B-TEST-*based* approaches have emerged especially for real-world problems [7, 21] that involve sequential decision-making, such as selecting a variation A/B/... The decision-making here

consists of assigning the *items* (patients, visitors, recommendations, ...) to the different variations (A/B/...) in order to evaluate the performance of each one (survival rate, average basket, click rate ...). During this *exploration*, it is assumed that the result, called *reward*, of each *assignment*, can be observed after a fixed period, to evaluate the performance of each variation. At the end of this exploration, the *user* can decide which variation should be implemented (i.e., during *production*) based on their performance. In [11], the authors highlighted conducting the A/B Testing in a sequential manner and without a random stopping rule to determine the completion of the experiment using a reinforcement learning policy (also known as a multiarmed bandit, MAB). The MAB is often formulated as the following problem: given a set of bandit “arms” (variation), each associated with a fixed but unknown reward probability distribution [12], an agent selects an arm to play at each iteration (when a visitor comes to the webpage), and receives a random reward variable (click, purchase, ...), sampled according to the selected arm’s distribution, independently of the previous actions. More formally, $\mathcal{A}$ is the space of actions (finite), $\mathcal{X}$ a set of rewards, at each iteration $t \in \mathbb{N}^+$, an agent select an arm $a_t \in \mathcal{A}$ and receive a reward $r_t \in \mathcal{X}$ where ξ_t is a noise centered ($\mathbb{E}[\xi] = 0$) and further conditionally sub-Gaussian such as:

$$r_t = \underbrace{f(a_t)}_{\text{reward funct.}} + \underbrace{\xi_t}_{\text{noise}}. \quad (1)$$

Note that f is initially unknown and can be the average (stationary or not) of the chosen arm reward a_t . If the agent chooses, at iteration t , a suboptimal variation, it suffers *simple regret* equal to the difference between the reward from the optimal variation a_* and the reward from the chosen variation a_t at iteration t . The goal of the agent is to minimize the cumulative sum of regret R_t at $t = T$, i.e. the end of the A/B-TEST : $R_T = T\mu_{a_*} - \sum_{a \in \mathcal{A}} N_a(T) \times \mu_a$ where $a_* = \operatorname{argmax}_{a \in \mathcal{A}} \mu_a$ (and μ_{a_*} the average of best arm) and $N_a(T)$ is the number of plays of an arm a at the end T of the A/B-TESTS. Thus, an efficient bandit policy must have an average regret less than the average regret of a random policy when $T \rightarrow \infty$. A decrease in regret implies that the agent selects arms that maximize gains, leading to an increase in average gain (e.g., average CTR) by the end of the test. Thus, one can study both regret (to minimize) and average gain (to maximize), the choice depending on the ability to observe rewards across all variations. A particularly useful version of MAB is the contextual multiarmed bandit (CMAB) [14], where at each iteration, before choosing an arm, the agent observes a d -dimensional context feature vector $c_t \in \mathbb{R}^d$ sampled from some unknown distribution. In that case the best arm is $a_* = \operatorname{argmax}_{a \in \mathcal{A}} \langle \theta_a, c_t \rangle$ with $\theta \in \mathbb{R}^d$ as the parameter of an arm. Then a reward becomes : $r_t = f(a_t, c_t) + \xi_t$. The c_t context vector encapsulates essential features of an item, such as age, origin, and gender, revealed before allocation choices. However, conventional CMAB approaches encounter challenges when c_t contains time series data. This inability to use time series to describe an item makes it impossible to employ CMABs in the context of A/B-TESTing related to e-commerce, where time series are commonly used to describe website visitors. However, deploying suboptimal

variations into production without knowing a stopping criterion in advance significantly hinders users from conducting A/B tests on their websites. A CMAB that utilizes time series as context would allow both the consideration of the evolving nature of website visitors and the encouragement of users to take risks to test variations. We propose in this paper a new approach by the inclusion of temporal data in c_t . Our methodology performs time series clustering based on the evolving features of visitors who have interacted with the original production page (Version A) in the past before conducting the test. This new approach not only recognizes the significance of time series but also introduces a nuanced preprocessing step, departing from traditional methodologies. By incorporating time series data and employing advanced clustering techniques, our method increases the average gain at the end of A/B-TEST (or decreases the regret). In addition, our innovative approach improves A/B testing practices by facilitating a deeper understanding of consumers behavior. Through the identification of distinct patterns in marketing, our methodology insights into tailoring experiments to specific audience segments. This contributes to a more nuanced interpretation of test. According to the user's needs, we propose two algorithms: DBA-CTREE-UCB and DBA-LINUCB, with significant improvements in terms of regret. Section 2 provides an overview of the state-of-the-art in CMAB methods, both with and without presegmentation. It also introduces the technique employed to address the temporal aspect of CMABs, and section 3 presents our two novel algorithms. Furthermore, Section 4 presents the experimental results obtained from various datasets. Finally, Section 5 concludes with a discussion.

2 State of the art

2.1 Classical MAB problem

The goal of the agent is to use knowledge from past observations to maximize long-term rewards : $a_{t+1} = F_t(a_1, r_1, \dots, a_t, r_t)$. To achieve this, the agent must determine and select systematically the arm with the highest average reward $a_\star = \operatorname{argmax}_{a \in \mathcal{A}} \mu_a$ as soon as possible, thus striking a balance between exploration (testing different arms) and exploitation (selecting the arm with the highest expected reward). Rather than focusing on cumulative rewards, which have no theoretical guarantee, theoretical analyses of bandit models focus on cumulative regret [12]. Observing convergent cumulative regret implies that the agent consistently makes the correct choices systematically after a time $t \in [1; T]$. As such, cumulative regret leads to theoretical bounds that are presented for many bandit algorithms in the literature. An in-depth technical analysis of the classical MAB was given in [13], where policies assuming only one best arm regardless of the item features (c_t) asymptotically reach a regret of $\mathcal{O}(\log T)$. However, to establish a theoretical bound on regret, it is essential to compare the performance against a reference model, often referred to as an oracle, which can learn from all the available data. Among the MABs classically used is UCB algorithm [12] which, for each arm $a \in \mathcal{A}$, constructs an adaptive upper confidence interval on the mean: $UCB(a, t) = \hat{\mu}_a(t) + \alpha_{\text{ucb}} \sqrt{\frac{2 \log(t)}{N_a(t)}}$ with $\alpha_{\text{ucb}} \in \mathbb{R}_+^*$ a positive

parameter and $N_a(t)$ is the number of selections of the arm a up to round t . At t , the U.C.B algorithm chooses $\operatorname{argmax}_{a \in \mathcal{A}} UCB(a, t)$. Other methods, such as UCB variants, Thompson Sampling, KL-UCB or EXP3 algorithms [20], can be used as alternatives to UCB. However, these algorithms have a regret bound that is strongly dependent on the joint distribution of the arms. The learning process takes longer when the means of the arms are close. One possible solution to maximize this difference is to leverage the partial information available before selecting an arm, which is defined as the *context*.

2.2 Contextual Multi Armed Bandit

In the CMAB problem, the agent receives partial information such as item features, referred to as context, before making a decision at each iteration. More formally, at each iteration t , the agent observes a particular item described by a d -dimensional feature vector $c_t \in \mathbb{R}^d$. The agent chooses an arm $a_t \in \mathcal{A}$ to apply to this item based on past contexts and rewards observed in previous iterations. Like in classical MAB, the agent cannot observe rewards from arms other than a_t . The context in the CMAB can be defined differently depending on how contexts are revealed and how assumptions are made about the reward function. In our case, each different context $\mathbf{c}_t$ is no longer a vector but a d time series of maximal size m , represented by a matrix $d \times m$. We distinguish the vector representation c_t from its matrix representation $\mathbf{c}_t$ which results in $r_t = f(a_t, \mathbf{c}_t) + \xi_t$ where $\mathbf{c}_t \in \mathbb{R}^{d \times m}$. This problem of context dimension was first studied in The Query-Ad-Algorithm [3]. The Query-Ad-Clustering algorithm achieves a regret of $\mathcal{O}(T^{1-\frac{1}{2+|\mathcal{C}|}+\epsilon})$, where $\mathcal{C}$ is the set of possible contexts and an ϵ positive constant. In the Query-Ad-Clustering algorithm, the reward estimates are accurate as long as the context partitions are similar to each other. However, when the context dimension is large, the regret bound becomes almost linear. This issue is addressed in [16], where the arm rewards are assumed to depend on an unknown subset of the context. It is demonstrated that the regret in this case depends only on the number of relevant groups [8] and requires a learning prestep; however, no details are given on how to achieve this group segmentation. Two approaches are possible in AB testing to define these groups: learning them during preprocessing (data collected before conducting the test) or online with contextual bandit.

Preprocessing approaches Preprocessing approaches assume the existence of natural groups, each having a Gaussian reward distribution [16], which can be determined online [15] or before the MAB allocation. The approach involves learning a set of groups $\mathcal{G}$ and a mapping group function association g before the A/B-TEST. Let's $g : \mathbb{R}^d \rightarrow \{0; 1\}^{|\mathcal{G}|}$ with $|\mathcal{G}| \in \mathbb{N}$ the number of possible groups. When a new visitor described by c_t is submitted to the agent, the function $g(c_t)$ classifies it into one of these groups ($g(c_t) = \tilde{g}_t$ with $\tilde{g}_t \in \mathcal{G}$) and then a non-contextual bandit strategy assigns it to an arm according to previous rewards. So partitioning between several groups limits regret when divergence is maximized. In [6] the authors propose the use of the CTREE-UCB algorithm

to construct the association function g using a conditional inference tree [9] (see Alg. 1 in Appendix) before applying several UCB bandits. Their partitioning involves collecting data from the original variation A and defining groups based on past visitors' features. Each of these groups has a reward distribution that can be modeled by a Gaussian distribution. Group detection here is a type of recursive partitioning method that involves the following basic steps:

- Step 1. Select the feature predictor that best separates different values of the reward distribution with a statistical p-value. The p-value error was corrected with a Bonferroni correction (a method to counter the problem of multiple comparisons).
- Step 2. This variable is split, and the data are divided into two datasets.
- Repeat steps 1 and 2 recursively until no further splits can be made based on predefined p-value rejection (according to the α_{CTREE} risk).
- A tree-like group partitioning model g is produced (see Fig 1).

To train the inference tree, CTREE-UCB uses the data present before starting the test (denoted as $\mathcal{L}$, collected on variation A). While the set of data from variation B is not observed, the authors in [6] show that in many cases of A/B-TESTS the groups observed on A are also observed in B. During the test, all new items are matched to a predefined group (from the tree). Each group can be supported by a noncontextual bandit. When a new item is presented to the agent, it is automatically assigned to a group based on its features, and an arm is assigned to it through the associated MAB, ensuring a satisfactory response time (millisecond time scale). This allows for a lower cumulative regret than other online learning methods, an interpretability due to the inference tree, and does not slow down the user experience (< 200 milliseconds), since the group learning step is performed only once and only non-contextual MABs are used afterwards. The alternative approach to the preprocessing step is learning the context function online, which is described in the following section.

Combinatory function Another possible approach is to learn the reward context function without any information before the test. A popular framework for contextual bandits is LinUCB proposed in [14], and variants [5] which assume a linear combination between the context and the d -parameters of each arm. LINUCB estimates the expected reward of each arm a as a linear regression of the context vector c_t , where $\theta_a \in \mathbb{R}^d$ is the regression coefficient of an arm reward function to be learned. We denote by α_{LINUCB} the parameter for the importance of exploration as $\alpha_{\text{LINUCB}} = 1 + \sqrt{(\log 2 / \delta_{CI}) / 2}$ where $1 - \delta_{CI}$ is the confidence interval. We assume that M is an invertible matrix; and denote by $M^{-1} \in \mathbb{R}^{d \times d}$ the updated weight, which can be interpreted as the covariance of the coefficient θ_a . Hence, LINUCB considers the upper confidence bound as $\hat{\theta}_a^T c_t + \alpha_{\text{LINUCB}} \sqrt{c_t^T M^{-1} c_t}$. The arm a_t selected is the one maximizing the upper confidence bound: $a_t = \operatorname{argmax}_{a \in \mathcal{A}} (\theta_a^T c_t + \alpha_{\text{LINUCB}} \sqrt{c_t^T M^{-1} c_t})$. LINUCB gives a regret in $\mathcal{O}(\sqrt{Td})$. Modified versions of this algorithm, such as SupLINUCB with kernel functions, are studied in [13] where the regret is $\mathcal{O}(\sqrt{Td})$. When

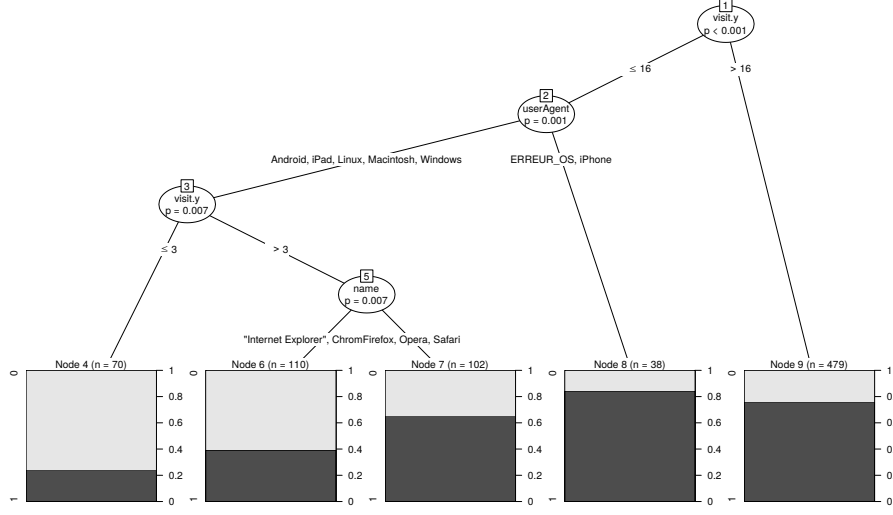


Fig. 1: Example of a conditional inference tree generated from the original page (AB Test dataset [6]). There are 5 groups with different CTR, determined by factors such as the number of past visits on the site (`visit.y`), user agent, and browser name. A separate UCB MAB will be employed for each identified group.

d is very large, inverting the matrix M can become computationally expensive and require a significant number of items. To handle nonlinear reward functions, recent works such as kernelized stochastic bandits [24] or deep neural networks [23], have considered the past selected arms and received rewards as training samples. [22] computes a gradient-based upper confidence bound with respect to the trained neural network strategy to select arms. However, when the context is a large vector, the aforementioned approaches need to work on batch style or become computationally expensive, which leads to a significant amount of regret.

2.3 Large-scale context

The algorithms detailed in the previous section have strong restrictions on the dimension of the context, where c_t must be a vector of numerical non correlated values. These restrictions limit the use of bandit algorithms when the features to be considered are time series ($\mathbf{c}_t \in \mathbb{R}^{d \times m}$). One solution to this problem is to flatten the time series of each temporal feature. One of the several problems with this approach is that if the size of feature vectors is large, it introduces significant variability in the bandit's performance, leading to a large amount of data to be collected. To address the problem of size context vectors, [4] proposed empirically building a LASSO-type statistical model and integrating it into a bandit problem. However, this approach requires that the features be independent of

each other, which is not the case when the features come from the same time series. In [18], the use of LSTM for handling time series data has the drawback of requiring a large number of visitors and frequent model retraining. This results in significant computational costs associated with deep learning methods and slows down the display of the test page, making it impractical for e-commerce applications. Considering temporal features is a difficult task without prior knowledge of the data. Additionally, we found no information in literature about A/B-TESTing contextual bandit algorithms that can adapt to data of variable size. The following section presents our contribution to address this issue.

3 Contribution

3.1 Illustrative dataset

We detail our approach using the AB tasty dataset 1 (owned by AB TASTY©). AB Tasty Dataset 1 comes from an AB testing platform that compares two versions of the same web page for an e-commerce site. The allocation of A or B to a visitor is randomized with a static allocation. Each visitor is assigned to a single variation until the end of the test (conducted over 15 days). If the visitor has made a purchase from the test page, a reward of 1 is assigned; otherwise, the value is 0. The data contains the history of visitor sessions generated before arriving on the test page: for each visitor, $\mathbf{c}_t$ includes the following information from the visitor’s first visit day on the site until the day he/she arrives on the tested page.

- **presence_time_serie**: series of binary values indicating for each day whether the visitor visited the site or not.
- **connexion_time_serie**: series of hours of connections (when the visitor arrived at the site).
- **time_spend_time_serie**: series of visitor’s session duration (milliseconds).

There are $n = 5156$ visitors, the shortest session is 2 days, and the longest session is 14 days.

3.2 Two new algorithms DBA-Ctree-Ucb and DBA-LinUCB for handling times series for A/B-Tests

Currently, there is no existing state-of-the-art method that effectively handles an evolving item context. However, the data presented prior to the beginning of the test could enable the transformation of this context, making it usable by a traditional context bandit method. This preprocessing step has already been employed in CTREE-UCB [6] to create an item segmentation that maximizes the difference between arm means and has demonstrated a significant decrease in cumulative regret. Our contribution proposes a model for replacing a visitor’s time series with clusters and demonstrates how this improves dynamic allocation compared to transforming series into averages. We introduce two novel extensions, DBA-CTREE-UCB and DBA-LINUCB, which complement the search for

context-dependent optimal variations with preprocessing utilizing DBA-DTW-kmeans. As we said in the introduction, domains such as e-marketing, and the recognition of evolving customer profiles are crucial for effectively targeting consumers, personalizing offers, and improving the user experience in an A/B test. Time series clustering can handle complex and multidimensional data, taking into account various variables such as purchase history, online interactions, geographical data, etc. By utilizing this method, marketers obtain a holistic view of consumer profiles and identify specific subgroups within the population. The clustering parameters can be adjusted based on the study objectives, such as the desired number of clusters or sensitivity to changes. To include time series clustering in a dynamic allocation algorithm, we propose two algorithms: DBA-CTREE-UCB and DBA-LINUCB. These algorithms improve the CTREE-UCB and LINUCB algorithms, respectively, which do not handle temporal aspects. It is important to note that the user is expected to obtain an original variation (such as web page A) and the historical data (such as logs) of the items that have undergone this original variation. These data, collected before their arrival on the test page, form learning clusters required for the DBA-LINUCB and DBA-CTREE-UCB algorithms. Each temporal series in the known items is replaced as a categorical feature: a cluster. The following section details our choice for time series clustering.

Choice of time Series Clustering Since the context of a user is represented by d time series, a solution could be to transform these series by d clusters. In our case, we decided to use a partitioning approach based on the similarity between items. Partitioning methods are computationally efficient and can handle large A/B-TEST datasets with ease. To implement such a method, we needed to set a similarity measure. Indeed, in time series clustering methods, if one wants to compare series with irregular sampling or of different sizes, particular attention must be given to the choice of similarity measure. There are many methods for measuring similarity between time series. The most well-known method is Euclidean distance, which involves calculating the sum of the squared distances between the corresponding elements of the considered sequences at each time step. If this distance is commonly admitted, it cannot be used in our case because it requires that the series be the same length. Length resampling would be problematic for a marketing application as it would risk losing the evolutionary aspect of the customer path. However, in our work, we place ourselves in a case where the observation of the context variables can be irregular. For example, one visitor may visit the site more frequently than another. The similarity measure DTW (dynamic time warping) is a metric between two series of different sizes widely recognized as relevant for many application domains. The DTW method matches the elements of the sequences by aligning them in a way that respects the total order of the sequence of values, without crossing the associations (see Alg. 2 in Appendix). DTW can be sensitive to noise, but this is not a problem in our web application where the data is not noisy. A warping path is constructed by computing the minimum cumulative distance between all possible pairs of points in the two-time series. It is usually calculated using the Euclidean dis-

tance, but other distance metrics can also be used. Once the warping path is obtained, the DTW distance is computed as the sum of the distances along the path. The objective function optimally solved by DTW corresponds to the minimization of the sum of the costs of the different associations. Fig 8 in Appendix illustrates the concept of the similarity measure DTW.

After defining the similarity metric used in our approach, we propose to use an averaging method based on the barycentre (Dynamic Barycenter Average, see Alg. 3 in Appendix) [17]. Once the series have been aligned using DTW and represented by their average series using DBA, the K-means divide the data into K clusters by similarity, where K is a predefined number (see Alg. 4 in Appendix). It assigns each series to the cluster whose average series is closest to it in terms of DTW similarity. Our number of clusters is based on the Silhouette score: it measures how similar a series is to each other. The Silhouette score, denoted as $S(i)$, is computed using the formula :

$$S(i) = \frac{b(i) - a(i)}{\max\{a(i); b(i)\}}$$

where $a(i)$ is the average DTW distance from the i th series to others in the same cluster, and $b(i)$ is the average distance from the i th series to series in the nearest cluster to which it does not belong. The optimal number of clusters is the one that maximizes the average Silhouette score. A higher Silhouette score indicates better-defined clusters. Our experiments have demonstrated that this is the most reliable indicator for determining the number of clusters (see Experimentation section). By combining these three components, the DBA-DTW-kmeans method identifies clusters of similar profiles, taking into account temporal variations and using a more flexible similarity measure. The Fig 2 shows an example with centroids obtained from DBA-DTW-Kmeans on the preprocessing step (774 visitors) on AB Tasty Dataset 1. We apply DBA-DTW-kmeans to `presence_time_serie`, `connection_time_serie` and `time_spend_time_serie` that describe visitors to AB Tasty dataset 1.

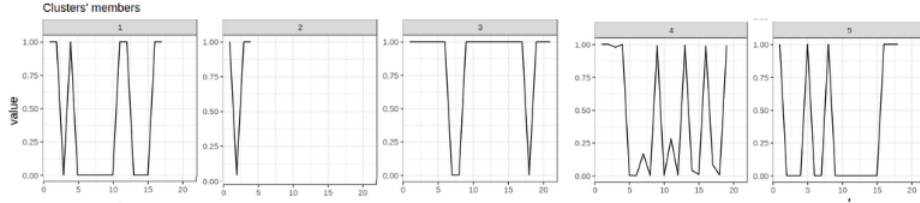


Fig. 2: The series represents 5 centroids based on the presence/non-presence per day series associated with visitors (from the first day of their visit to the site to the day they appeared on the page being tested). If the visitor does not log on during the day, the series is set to 0.

Dynamic allocation Each time series is replaced by the centroid to which it is closest. For DBA-CTREE-UCB the tree model g is learned from $\mathcal{L}$, where each time series has been substituted with the number of clusters to which it belongs. The second step of these two algorithms corresponds to the online A/B-TEST step involving CTREE-UCB and LINUCB. Cluster replacement can then be directly utilized by a contextual bandit model. Consequently, in the DBA-LINUCB/DBA-CTREE-UCB version, the allocation is down according to a LINUCB/CTREE-UCB modelling (see Alg. 5 and Alg. 6 in Appendix). The global schema of our idea is drawn in Fig 9 in Appendix. The offline part is common to both methods: the past log is used to generate the clusters. An additional step for DBA-CTREE-UCB also creates the segmentation tree that will be used in the online part. The online part corresponds to the start of the test. The time series of new items are replaced by the nearest cluster, and the allocation of variation depends on the bandit strategy used (according to the segmentation tree g for DBA-CTREE-UCB and according to the upper bound of a linear regression for DBA-LINUCB). The DBA-LINUCB algorithm uses a linear approach to estimate the parameters between clusters and potential rewards. This approach assumes that this reward function can be approximated by a linear function. The DBA-CTREE-UCB divides the cluster sets into distinct groups, enabling the capture of more complex and nonlinear function links between the context and rewards. It evaluates each group and identify optimal variation according to each group by the UCB strategy. This approach facilitates faster exploration when one or more groups demonstrate sensitivity to the test while avoiding unnecessary slowdown in cases where the test would yield no changes for some items (e.g. Fig 3).

4 Experiments

Here, we present the results of the DBA-CTREE-UCB and DBA-LINUCB models on real-world datasets. We compare our algorithms with CTREE-UCB [6] and LINUCB [1] which require transforming each context time series by taking the average of its values. These experiments observe whether clustering (which is defined by a partial observation of the data) significantly reduces regret. The choice of the number of clusters is first evaluated using the Silhouette index [19]. All experiments are reproducible in our R package from our GitHub repository⁴.

AB Tasty Dataset 1 We present the results on the dataset introduced in section 3.1. The available payoffs are those associated with the pages presented in the case of a static allocation, so observing the regret requires a k-nearest neighbors replacement technique : we replaced the missing values by sampling from the visitors at a minimal distance (<10% of the sequence for each series). If no similar visitor profile exists, a reward is randomly drawn from the alternative page dataset. The "k-nearest neighbors" method, employed to replace missing data, keeps the correlation between the time series and the corresponding generated

⁴ <https://github.com/manuclaeys/banditWithR>

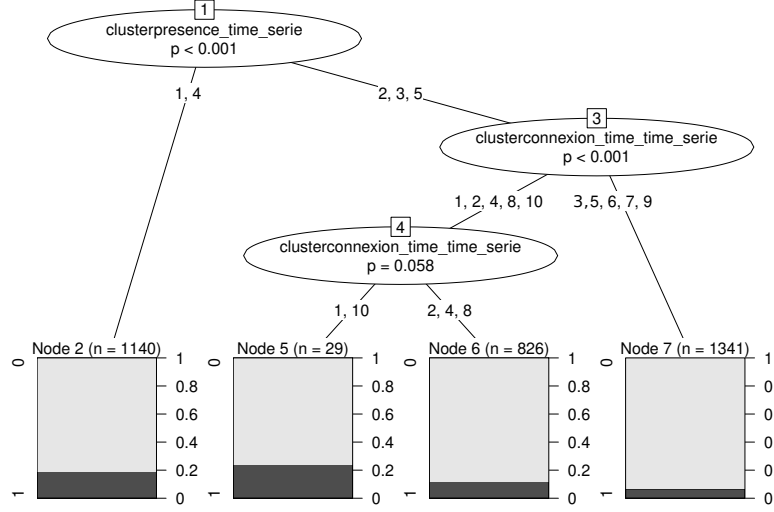


Fig. 3: Preprocessing tree obtained by DBA-CTREE-UCB on AB TASTY dataset 1. Each leaf represents a click rate. 4 groups are identified from the pre-processing dataset.

rewards. Unlike predefined theoretical replacement models, this method does not rely on such models, which may introduce a strong dependency between missing values and their replacements in small datasets; however, this is not the case in our experiment.

Number of clusters and regret The cumulative regret of DBA-CTREE-UCB and DBA-LINUCB is influenced by the choice of number of clusters, as depicted in the three graphs of Fig 4. Our method chooses to parameterize the number of clusters based on the Silhouette index (here 5, 5, 10 for the three series). When employing DBA-CTREE-UCB, the use of an inference tree to partition visitors into subgroups introduces the possibility of grouping clusters with no significant differences (statistically) if the number of clusters is set too large (see Fig 3). On the other hand, for DBA-LINUCB, the learning time and regret increase linearly with the number of clusters. Consequently, choosing an inappropriate value will lead to a longer learning process and result in a greater level of regret. The centroids generated by DBA-CTREE-UCB and DBA-LINUCB must be interpretable from a marketing perspective by the user. Those presented in Fig. 2 represent, for example, visitor patterns such as: the regular visitor (3), those prospecting for a product (1, 4, and 5) with varying intervals between visits, and those who do not return (2). By coupling with other types of series DBA-CTREE-UCB identifies "high-potential" or "low-potential" visitors. In the generated tree (see Fig. 3), DBA-CTREE-UCB identifies 4 visitor groups, each with a varying CTR. New visitors placed in these groups had an independent allocation by UCB

policy, and the user was able to identify at the end of the test which variation was better for each group. It should be noted that generally, groups with very low CTR are usually less sensitive to the test. For example, visitors in group 7 (see Fig. 3) showed no preference between variations. This visitor group was highly represented in the traffic, and its 'isolation' helped accelerate learning for the other groups.

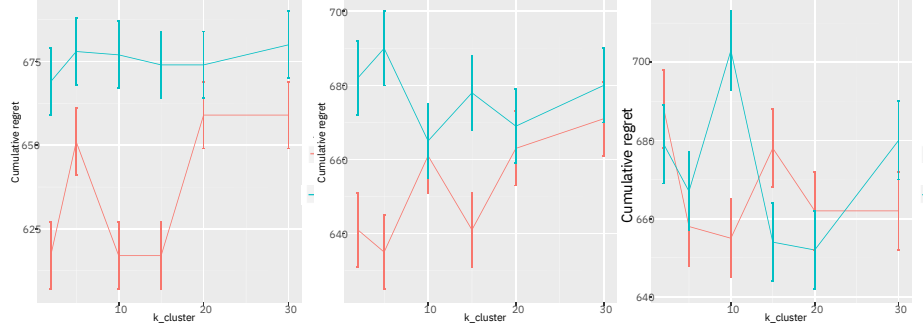


Fig. 4: Cumulative regret obtained by DBA-CTREE-UCB (red) and DBA-LINUCB (light blue) over number of clusters for AB Tasty dataset 1 on **presence_time** (left), **connexion_time** (middle) and **time_spend** (right). For each graph, the algorithms are trained using only one of the three series as feature context information. In comparison, Silhouette's indexes suggest the following settings (**presence_time** = 5, **connexion_time** = 5, **time_spend** = 10)

The evolution of cumulative regret according to time for all methods is presented in Fig. 5. It is observed that the gap between DBA-CTREE-UCB and the other methods widens during learning. The cumulative regret (to minimize) according to clusters setting for DBA-CTREE-UCB and DBA-LINUCB is shown in Fig. 4 and confirms that the silhouette index is a good indicator for choosing the number of clusters before starting the test. The average click-through rates (to maximize) as a function of the number of clusters are also referenced in Tab 1 for comparing all methods with different settings (number of clusters, web page used for preprocessing). The lower performance of DBA-LINUCB compared to DBA-CTREE-UCB can be explained by the fact that if the number of clusters is large, DBA-LINUCB will require more data than DBA-CTREE-UCB. The differences in terms of regret may seem small, but as the objective of the test is a click to buy, the probability of success is relatively low, regardless of the variation displayed. The average click rate at the end of A/B-TEST was 14.14% for DBA-CTREE-UCB, 12.79% for CTREE-UCB and < 12% for DBA-LINUCB, LINUCB and UNIFORM (see Table 1).

Complementary experiment : AB Tasty 2 We provide a complementary experiment on another AB Tasty dataset (AB Tasty dataset 2). These data have the

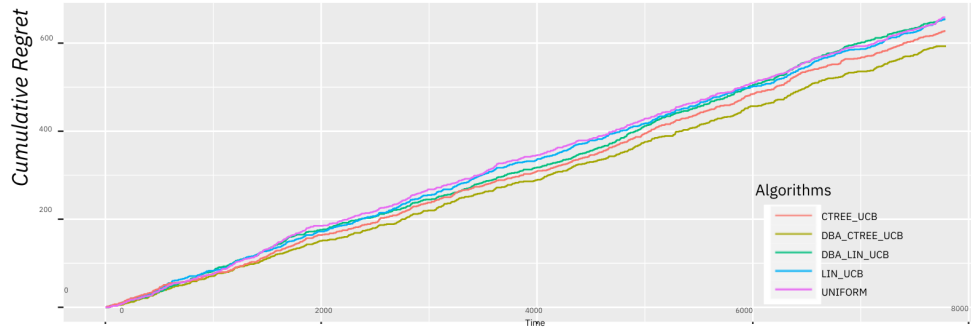


Fig. 5: Cumulative Regret UCB, LINUCB, CTREE-UCB, DBA-CTREE-UCB and DBA-LINUCB on AB TASTY dataset 1

same type of time series and come from a video streaming website. The objective of the AB Test is to increase the click-through rate of the tested page. To learn the clusters and groups, we test several configurations. One uses 30% of the visitors (1591), and the other uses all the visitors (5306). The reader can see the results in the table 1, in particular the interest of the two new approach proposed to increase the average click rate at the end of the experiment. DBA-CTREE-UCB identified 2 visitor groups (see Fig 6) where CTREE-UCB did not identify any groups. The Silhouette index suggested setting 10,5 and 10.

Localization Data for Posture Reconstruction In this experiment, we assess the performance of DBA-CTREE-UCB and DBA-LINUCB in a non-e-commerce setting using the 'Localization Data for Posture Reconstruction' dataset [10]. The dataset comprises 164860 positional measurements from five patients, each represented by three time series ($V1, V2, V3$) capturing displacements along the x, y, and z axes. The objective is to differentiate between 'sitting' and 'sitting on the floor' activities based on these measurements. Our A/B test categorizes activities as A ('sitting on the floor') and B ('sitting'), with rewards assigned for successful detection. We frame this as a classification problem, aiming to identify patient activities accurately without imputation, unlike traditional e-commerce datasets. The study involves 1000 items and various bandit configurations. The results are on Tab 2. The parameter settings are according to the max Silhouette index: $K_{V1} = 10$, $K_{V2} = 10$, $K_{V3} = 10$. The regret comparison is shown in Figure 7 provides additional evidence to support the effectiveness of the method effectiveness for signal-type series. For this dataset, DBA-LINUCB achieves the best performance. The lower performance of DBA-CTREE-UCB can be explained by the fact that the sensors are not strongly correlated with each other, and the choice of sensor used for learning strongly influences the algorithm's performance.

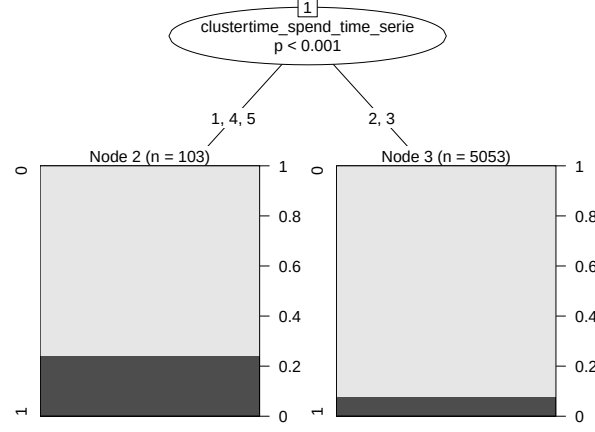


Fig. 6: Preprocessing tree obtained by DBA-CTREE-UCB on AB TASTY dataset 2. Each leaf represents a click rate. 2 groups are identified from the pre-processing dataset.

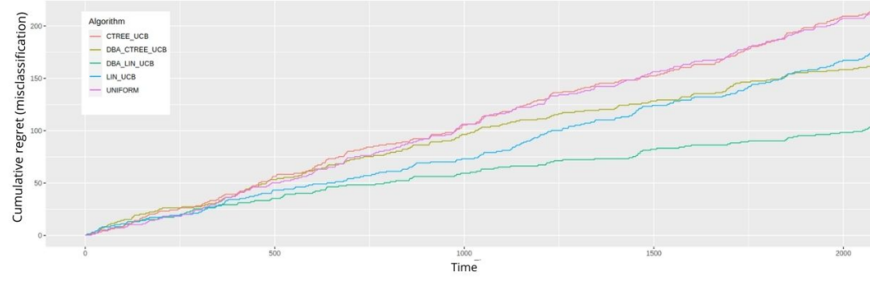


Fig. 7: Cumulative regret UCB, LINUCB, CTREE-UCB, DBA-CTREE-UCB and DBA-LINUCB for Position Dataset

5 Discussion of results and conclusion

The choice of cluster number K , based on the Silhouette index, indicates that the algorithm is suitable for parameter selection. We tested 6 other quality indices, none of which yielded such results. One crucial aspect of our method is the reduction in context size, which leads to a significant decrease in computation time. Since the cluster identification component is executed offline, it does not decrease the efficiency of the online A/B-TEST test step. The DBA-CTREE-UCB method enables the grouping of previously learned clusters if the reward distributions (on variation "A") are statistically identical. Therefore, setting the number of clusters too large has minimal impact on the regret, as clusters with identical

	AB Tasty Dataset 1		AB Tasty Dataset 2	
	Conf _{50,70}			
DBA-CTREE-UCB	$\mathcal{L} = P_1$	$\mathcal{L} = P_2$	$\mathcal{L} = P_1$	$\mathcal{L} = P_2$
Nb clusters				
3;5;5	13.80±1.1%	12.56±1.1%	9.53±1%	9.53±1%
5;5;10	14.14±1.2%	13.47±1.2%	9.53±1%	9.53±1%
10;5;10	13.67±1.2%	12.82±1%	9.92±1%	8.66±1%
DBA-LINUCB				
Nb cluster				
3;5;5	11.61±1.1%		8.39±1%	
5;5;10	11.89±1.2%		8.51±1%	
10;5;10	11.81±1.2%		9.19%	
CTREE-UCB	12.79%±1%	11.47%±1%	9.07%	9.09%
LINUCB	11.56%±1%		8.57±1%	
UNIFORM	11.49%±1%		8.26%±1%	

Table 1: Average click rate at the end of the test according to different settings for AB Tasty dataset 1 and 2

	Localization Dataset		
	Conf _{50,70}		
DBA-CTREE-UCB	$\mathcal{L} = V1$	$\mathcal{L} = V2$	$\mathcal{L} = V3$
Nb cluster			
5;5;10	68.9%±3%	44.1%±4%	43.6%±3%
10;10;10	69.8%±3%	58.6%±4%	66.8%±3%
10;15;15	68.8%±4%	57.3%±4%	35.6%±4%
DBA-LINUCB			
Nb cluster			
5;5;10	76%±3%		
10;10;10	81.7%±3%		
10;15;15	74.6%±3%		
LINUCB	66.2%±2%		
CTREE-UCB	33.7%±3%	32.2±3%	31.5±4%
UNIFORM	32.5%±1%		

Table 2: Average classification rate at the end of the test according to different settings for Localization Dataset

reward distributions are grouped together. Our experiments also demonstrated that DBA-CTREE-UCB and DBA-LINUCB facilitate business interpretation of clusters. For instance, separating a "perfect prospect" from a "visitor who arrived by mistake" can be challenging. Their visits are nearly short before they reach the test page. By combining different types of clusters (one based on presence and the other on time spent on the site) and predicting their click probabilities, we can differentiate between these two profiles and determine the most suitable variation. DBA-CTREE-UCB/DBA-LINUCB appears to be a more advantageous method than CTREE-UCB/LINUCB in terms of regrets. Clusters allow for the construction of more homogeneous groups in terms of reward distribution rather than relying solely on series averages. Separating groups based on series mean values makes learning highly sensitive to extreme values, which the clustering model avoids. However, DBA-CTREE-UCB requires a correlation between the earnings of different variations: the earnings distribution of a group, whether on variation A or B, follows the same distribution (with the same variance), but the means may differ. In a further work we will show how this pre-processing step has helped the user to create more personalized variation B by generative DNN.

Bibliography

- [1] Agarwal, A., Dudík, M., Kale, S., Langford, J., Schapire, R.E.: Contextual Bandit Learning with Predictable Rewards. ArXiv e-prints (feb 2012)
- [2] Auer, P., Cesa-Bianchi, N., Fischer, P.: Finite-time analysis of the multi-armed bandit problem. Machine Learning **47**(2), 235–256 (May 2002)
- [3] Avadhanula, V., Colini-Baldeschi, R., Leonardi, S., Sankararaman, K.A., Schrijvers, O.: Stochastic bandits for multi-platform budget optimization in online advertising. CoRR (2021)

- [4] Bastani, H., Bayati, M.: Online decision-making with high-dimensional covariates. *SSRN Electronic Journal* (01 2015)
- [5] Bietti, A., Agarwal, A., Langford, J.: A contextual bandit bake-off. *JLMR* (2021)
- [6] E. Claeys, P. Gancarski, M. Maumy-Bertrand, and H. Wassner: Dynamic allocation optimization in a/b-tests using classification-based preprocessing. *IEEE TKDE* **35**(1), 335–349 (2021)
- [7] Fabijan, A., Dmitriev, P., Arai, B., Drake, A., Kohlmeier, S., Kwong, A.: A/b integrations: 7 lessons learned from enabling a/b testing (2023)
- [8] Gentile, C., Li, S., Kar, P., Karatzoglou, A., Zappella, G., Etrue, E.: On context-dependent clustering of bandits. In: *Proceedings of the 34th International Conference on Machine Learning* (2017)
- [9] Hothorn, T., Hornik, K., Zeileis, A.: Unbiased recursive partitioning: A conditional inference framework. *Journal of Comput and Graphical Statistics* **15**, 651–674 (09 2006)
- [10] Kaluza, B., Mirchevska, V., E. Dovgan, M.L., Gams, M.: UCI machine learning repository, an agent-based approach to care in indep. living (2010)
- [11] Kaufmann, E., Cappé, O., Garivier, A.: On the Complexity of A/B Testing. *ArXiv e-prints* (may 2014)
- [12] Lai, T., Robbins, H.: Asymptotically efficient adaptive allocation rules. *Advances in Applied Mathematics* **6**(1), 4 – 22 (1985)
- [13] Lattimore, T., Szepesvári, C.: *Bandit algorithms* (2020)
- [14] Li, L., Chu, W., Langford, J., Schapire, R.E.: A contextual-bandit approach to personalized news article recommendation pp. 661–670 (2010)
- [15] Mahadik, K., Wu, Q., Li, S., Sabne, A.: Fast distributed bandits for online recommendation systems. *Proceedings of the 34th ACM International Conference on Supercomputing* (2020)
- [16] Maillard, O.A., Mannor, S.: *Latent Bandits*. (Jan 2014), extended version of the paper accepted to ICML 2014.
- [17] Petitjean, F., Ketterlin, A., Gancarski, P.: A global averaging method for dtw, with applications to clustering. *Pattern Recognition* **44**, 678– (03 2011)
- [18] Rotman, M., Wolf, L.: Energy regularized rnns for solving non-stationary bandit problems (2023)
- [19] Rousseeuw, P.: Rousseeuw, p.j.: *Silhouettes: A graphical aid to the interpretation and validation of cluster analysis*. *JCAM* (1987)
- [20] Thompson, W.R.: On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. *Biometrika* (1933)
- [21] Xu, D., Yang, B.: On the advances and challenges of adaptive online testing. *WSDM’22* (2022)
- [22] Zhang, Z., Yang, J., Ji, X., Du, S.S.: Variance-aware confidence set: Variance-dependent bound for linear bandits and horizon-free bound for linear mixture MDP. *CoRR* (2021)
- [23] Zhou, D., Li, L., Gu, Q.: Neural contextual bandits with upper confidence bound-based exploration. *CoRR* (2019)
- [24] Zhou, X., Ji, B.: On kernelized multi-armed bandits with constraints. In: *Advances in Neural Information Processing Systems*. Curran Associates, Inc. (2022)

Appendix

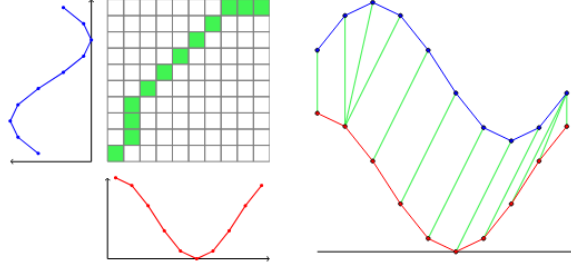


Fig. 8: Similarity of two series by DTW. (left) Matrix and optimal alignment path calculated by DTW. (right) Resulting alignment of the two sequences

Algorithm 1 CTREE algorithm

Require:

- 1: – An Alpha risk hyperparameter $\alpha_{\text{CTREE}} \in]0, 1[$
 – A dataset of features $c_t \in \mathcal{C}$ and response $r_t \in \mathcal{X}$.
- 2: Calculate the test statistics $stat_{j_0}$ for the observed data
- 3: Permute the observation in the node
- 4: Calculate $stat$ for all permutations
- 5: Calculate the p -values (from the number of test statistics $stat$, where $|stat| > |stat_0|$)
- 6: Correct p -values for multiple testing according a Bonferroni correction.
- 7: **if** H_0 (no statistical difference between distributions) are not rejected (p -value $> \alpha_{\text{CTREE}}$ for all X_j) **then return**
- 8: Select feature X_j^* with the strongest association (smallest p -value)
- 9: Search for the best split of X_j^* (maximize test statistic $stat_0$) and partition data
- 10: Apply CTREE to both of the new partitions

Output: A hierarchical partitioning g function

Algorithm 2 Algorithm DTW

Require:

- 1: – A sequence $S1 = \langle s1_1, \dots, s1_{|S1|} \rangle$.
- A sequence $S2 = \langle s2_1, \dots, s2_{|S2|} \rangle$.
- A cost matrix **costmat** of size $|S1| \times |S2|$ initialized to 0.
- 2: **costmat**[1, 1] $\leftarrow \delta(s1_1, s2_1)$
- 3: **for** $i \in \{2, \dots, |S1|\}$ **do**
- 4: **costmat**[i , 1] = **costmat**[$i - 1$, 1] + $\delta(s1_i, s2_1)$
- 5: **for** $j \in \{2, \dots, |S2|\}$ **do**
- 6: **costmat**[1, j] = **costmat**[1, $j - 1$] + $\delta(s1_1, s2_j)$
- 7: **for** $i \in \{2, \dots, |S1|\}$ **do**
- 8: **for** $j \in \{2, \dots, |S2|\}$ **do**
- 9: **costmat**[i , j] = $\delta(s1_i, s2_j) + \min \begin{Bmatrix} \text{costmat}[i - 1, j] \\ \text{costmat}[i, j - 1] \\ \text{costmat}[i - 1, j - 1] \end{Bmatrix}$

Output: **costmat**

Algorithm 3 Algorithm D.B.A

Require:

- 1: – A learning set $\mathcal{L}$.
- $S_{\text{aver}} = \langle s_{\text{aver},1}, \dots, s_{\text{aver},|S_{\text{aver}}|} \rangle$ the initial average sequence.
- $S1 = \langle s1_1, \dots, s1_{|S1|} \rangle$ the first sequence of the set $\mathcal{L}$.
- ...
- $S_n = \langle sn_1, \dots, sn_{|S_n|} \rangle$ the last sequence of the set $\mathcal{L}$.
- 2: Let m the maximal size of all sequences in $\mathcal{L}$.
- 3: Let *assocTab* an empty table of size $|S_{\text{aver}}|$ contain in each cell l all the elements associated with l in S_{aver} .
- 4: Let **costmat** $\in \mathbb{R}^{m \times m}$ the cost matrix for DTW (See Alg 2).
- 5: **for** $Si \in \mathcal{L}$ **do**
- 6: **costmat** $\leftarrow DTW(S_{\text{aver}}, Si)$
- 7: $l \leftarrow |Si_{\text{aver}}|$
- 8: $j \leftarrow m$
- 9: **while** $l \geq 1$ or $j \geq 1$ **do**
- 10: $\text{assocTab}[l] \leftarrow \text{assocTab}[l] \cup si_j$
- 11: $(l, j) \leftarrow \text{second}(\text{costmat}[l, j])$
- 12: **for** $l \in 1 \dots |S_n|$ **do**
- 13: $S_{\text{aver}}^+ = \text{barycenter}(\text{assocTab}[l])$

Output: S_{aver}^+

Note that D.B.A. utilizes a matrix that returns for each cell (access with first()) function all elements associated with this cell (access with second()).

Algorithm 4 Pseudo code of K-means D.B.A DTW

Require:

- 1: – A number of desired cluster K
- A learning set $\mathcal{L} = \{S_1, \dots, S_n\}$ of n sequence.
- $Iter$ maximal number of iteration
- 2: Randomly choose a cluster for each time series
- 3: **while** $Iter$ **do**
- 4: Calculate the mean for each cluster according to D.B.A (See Alg. 3)
- 5: Assign each sequence to the closest centroid according do DTW (See Algo. 2)

Output: A set of centroid

Algorithm 5 Algorithm DBA-CTREE-UCB

Require:

- 1: – A learning set $\mathcal{L}_{\text{pre-test_A}}$
- A set of T items $\mathcal{L}_{\text{test_A_B}}$ to be tested.
- vectClust a d dimensional vector of desired number of clusters for each d type of series.
- An accepted risk $\alpha_{\text{CTREE}} \in [0, 1]$.
- A positive parameter $\alpha \in \mathbb{R}^+$.
- 2: Init **matrixClusters** an empty matrix of all centroides
- 3: **for** Each time series of type $j \in d$ in $\mathcal{L}_{\text{pre-test_A}}$ **do**
- 4: Subset j -type series in $\mathcal{L}_{\text{pre-test_A},j}$
- 5: **matrixClusters** $\leftarrow$ K-means-DBA-DTW($\mathcal{L}_{\text{pre-test_A},j}$), add $\text{vectClust}[j]$ centroids
- 6: Generate a $k()$ function that return a d -vector of the closest clusters according to **matrixClusters**
- 7: **for** Each item $\mathbf{c}_t \in \mathcal{L}_{\text{pre-test_A}}$ **do**
- 8: $c_t \leftarrow k(\mathbf{c}_t)$
- 9: Generate a $g()$ function that return a leaf group association according to CTREE($\mathcal{L}_{\text{pre-test_A}}, \epsilon$)
- 10: **while** $\mathbf{c}_t \sim \mathcal{L}_{\text{test_A_B}}$ **do**
- 11: Receive an item $\mathbf{c}_t$ from $\mathcal{L}_{\text{test_A_B}}$
- 12: Assign $\mathbf{c}_t$ to a group according $g \circ k(\mathbf{c}_t)$
- 13: **if** $N_{a, g \circ k(\mathbf{c}_t)}(t) = 0$ **then**
- 14: $a_t = a$
- 15: $a_t = \operatorname{argmax}_{a \in \mathcal{A}} \{ \hat{\mu}_{a, g \circ k(\mathbf{c}_t), t} + \alpha \sqrt{\frac{2 * \log(\sum_{a \in \mathcal{A}} N_{a, g \circ k(\mathbf{c}_t)}(t))}{N_{a, g \circ k(\mathbf{c}_t)}(t)}} \}$
- 16: Assign the item $\mathbf{c}_t$ to arm a_t
- 17: Receive a reward $r_{\mathbf{c}_t, a_t}$
- 18: Update $\hat{\mu}_{a, g \circ k(\mathbf{c}_t), t}$ and $N_{a, g \circ k(\mathbf{c}_t)}(t)$
- 19: Remove $\mathbf{c}_t$ from $\mathcal{L}_{\text{test_A_B}}$

Output: A sequence of T arm choices and rewards

Algorithm 6 Algorithm DBA-LINUCB

Require:

- 1: – A learning set $\mathcal{L}_{\text{pre-test_A}}$
- A set of T items $\mathcal{L}_{\text{test_A_B}}$ to be tested.
- vectClust a d dimensional vector of desired number of clusters for each d type of series.
- A positive parameter $\alpha \in \mathbb{R}^+$.
- 2: Init **matrixClusters** an empty matrix of all centroides
- 3: **for** Each time series of type $j \in d$ in $\mathcal{L}_{\text{pre-test_A}}$ **do**
- 4: Subset j -type series in $\mathcal{L}_{\text{pre-test_A},j}$
- 5: **matrixClusters** $\leftarrow$ K-means-DBA-DTW($\mathcal{L}_{\text{pre-test_A},j}$), add $\text{vectClust}[j]$ centroids
- 6: Generate a $k()$ function that return a d -vector of the closest clusters according to **matrixClusters**
- 7: Define $D = |\mathcal{K}|$ as the number of possible cluster (hot-one encoding)
- 8: **for** $a \in \mathcal{A}$ **do**
- 9: Initialize $A_a = I_D$, the identity matrix of size D by D of arm a
- 10: Initialize $b_a = 0_D$, a null vector of size D
- 11: **while** $\mathbf{c}_t \sim \mathcal{L}_{\text{test_A_B}}$ **do**
- 12: Receive an item $\mathbf{c}_t$ from $\mathcal{L}_{\text{test_A_B}}$
- 13: **for** $a \in \mathcal{A}$ **do**
- 14: Update parameters $\hat{\theta}_{t,a} = A_a^{-1}b_a$
- 15: **end for**
- 16: Choose $a_t = \text{argmax}_{a \in \mathcal{A}} \hat{\mu}_{a,t} = k(\mathbf{c}_t)^T \hat{\theta}_{a,t} + \alpha \sqrt{k(\mathbf{c}_t)^T A_a^{-1} k(\mathbf{c}_t)}$
- 17: Receive a reward $r_{k(\mathbf{c}_t), a_t}$
- 18: Update $A_{a_t} = A_{a_t} + k(\mathbf{c}_t)k(\mathbf{c}_t)^T$
- 19: Update $b_{a_t} = b_{a_t} + r_{\mathbf{c}_t, a_t} k(\mathbf{c}_t)$

Output: A sequence of arm choices and rewards

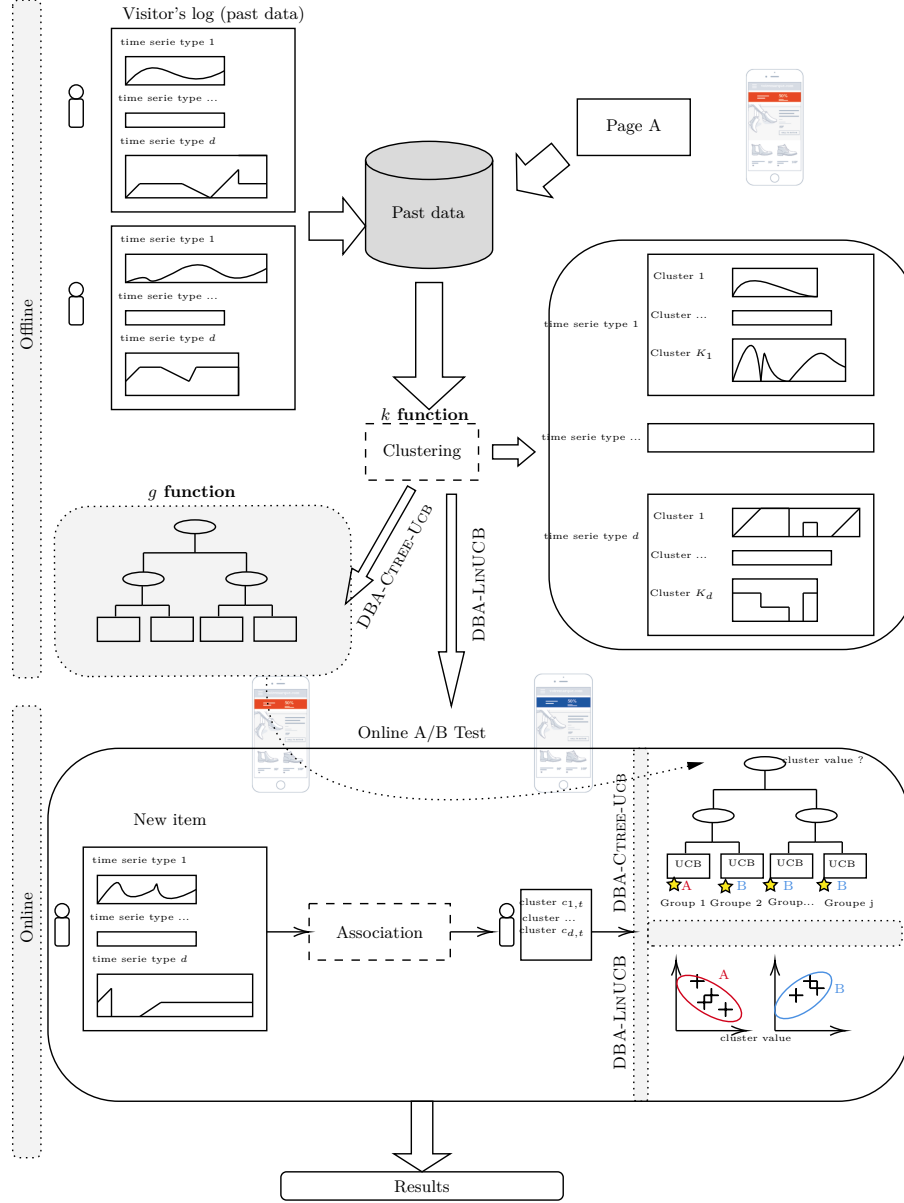


Fig. 9: Offline/online approach to integrate time series in A/B tests

DBA-Ctree-Ucb convergence

If a group of items $\tilde{g}$ have a $\sigma_{\tilde{g}}$ -subgaussian distribution of reward for a s number of trials with $s \leq n$ and a given $1 - \delta$ probability, the real average reward of an arm a is bounded by:

$$\frac{1}{s} \sum_{t=1}^s X_{a,t,\tilde{g}} \leq \mu_{a,\tilde{g}} + \sqrt{\frac{2\sigma_{\tilde{g}}^2 \log \frac{1}{\delta}}{s}} \quad (2)$$

with traditionally $\delta = \frac{1}{n^2}$ [2].

We assume the case where a prediction function assign items to a group $\tilde{g}$ where it's reward follows a $\sigma_{\tilde{g}}$ -gaussian distribution regardless of the arm applied. The experiences in the next section indicate that this hypothesis can be confirmed in practice. Define a reward at iteration t : $X_t = X_{A_t,\tilde{g}}$ and $N_{A_t,\tilde{g}}(t)$ the number of time when the arm A_t in group $\tilde{g}$ have been played. By definition of regret:

$$R_{n,\tilde{g}} = \sum_{a \in \mathcal{A}} \Delta_{a,\tilde{g}} \mathbb{E}[N_{a,\tilde{g}}(n)] \quad (3)$$

With $\Delta_{a,\tilde{g}} = \mu_{a^*,\tilde{g}} - \mu_{a \neq a^*,\tilde{g}}$. The regret increase of $\Delta_{a,\tilde{g}}$ when

- (a) $\text{U.C.B}_{a \neq a^*}(\tilde{g}, t) > \mu_{a^*,\tilde{g}}$
- (b) $\text{U.C.B}_{a=a^*}(\tilde{g}, t) < \mu_{a^*,\tilde{g}}$

Let's $G_{a,\tilde{g}}$ the event of (a) or (b) define by a constant $u_{a,y}$:

$$G_{a,\tilde{g}} = \{\mu_{a^*,\tilde{g}} < \min_{t \in [n]} \text{U.C.B}_{a=a^*}(\tilde{g}, t)\} \cap \{\hat{\mu}_{a,\tilde{g},u_i} + \sigma_{\tilde{g}} \sqrt{\frac{1}{u_a} \log \frac{1}{\delta}} < \mu_{a^*,\tilde{g}}\}$$

where if $G_{a,\tilde{g}}$ happens, the regret converge to a finite value and so if $G_{a,\tilde{g}}$ occur, then $N_{a,\tilde{g}}(n) \leq u_{i,\tilde{g}}$. Let $G_{a,\tilde{g}}^C$ the complementary event.

$$\begin{aligned} \mathbb{E}[N_{a,\tilde{g}}(n)] &= \mathbb{E}[\mathbb{I}\{G_{a,\tilde{g}}\} N_{a,\tilde{g}}(n)] + \mathbb{E}[\mathbb{I}\{G_{a,\tilde{g}}^C\} N_{a,\tilde{g}}(n)] \\ &\leq u_{a,\tilde{g}} + \mathbb{P}[G_{a,\tilde{g}}^C] n \end{aligned}$$

Bound of complementary event $G_{a,\tilde{g}}^C$ is done in two parts:

$$\begin{aligned} G_{a,\tilde{g}}^C &= \left\{ \mu_{a^*,\tilde{g}} \geq \min_{t \in [n]} \text{U.C.B}_{a=a^*}(\tilde{g}, t) \right\} \cup \left\{ \hat{\mu}_{a,\tilde{g},u_a} + \sigma_{\tilde{g}} \sqrt{\frac{2}{u_a} \log \frac{1}{\delta}} \geq \mu_{a^*,\tilde{g}} \right\} \end{aligned}$$

$$\begin{aligned}
\{\mu_{a^*,\tilde{g}} \geq \min_{t \in [n]} \text{U.C.B}(\tilde{g}, t)\} &\subset \cup_{s \in [n]} \{\mu_{a^*,\tilde{g}} \geq \hat{\mu}_{a^*,\tilde{g},s} \\
&\quad + \sigma_{\tilde{g}} \sqrt{\frac{2}{s} \log \frac{1}{\delta}}\} \\
\mathbb{P}[\mu_{a^*,\tilde{g}} \geq \min_{t \in [n]} \text{U.C.B}_{a=a^*}(\tilde{g}, t)] &\leq \sum_{s=1}^n \mathbb{P}[\mu_{a^*,\tilde{g}} \geq \hat{\mu}_{a^*,\tilde{g},s} \\
&\quad + \sigma_{\tilde{g}} \sqrt{\frac{2}{s} \log \frac{1}{\delta}}] \\
&\leq n\delta
\end{aligned}$$

And so $\mathbb{P}[G_{a,\tilde{g}}^C] \leq n\delta + \mathbb{P}[\hat{\mu}_{a,\tilde{g},u_a} + \sqrt{\frac{2}{u_a} \log \frac{1}{\delta}} \geq \mu_{a^*,\tilde{g}}]$. We remind that $\Delta_{a,\tilde{g}} = \mu_{a^*,\tilde{g}} - \mu_{a,\tilde{g}}$. Assume a positive constant c where $\Delta_{a,\tilde{g}} - \sigma_{\tilde{g}} \sqrt{\frac{2}{u_a} \log \frac{1}{\delta}} \geq c\Delta_{a,\tilde{g}}$:

$$\begin{aligned}
\mathbb{P}[\hat{\mu}_{a,\tilde{g},u_a} + \sigma_{\tilde{g}} \sqrt{\frac{2}{u_a} \log \frac{1}{\delta}} \geq \mu_{a^*,\tilde{g}}] &= \mathbb{P}[\hat{\mu}_{a,\tilde{g},u_a} \\
&\quad + \sigma_{\tilde{g}} \sqrt{\frac{2}{u_a} \log \frac{1}{\delta}} \geq \Delta_{a,\tilde{g}} \\
&\quad + \mu_{a,\tilde{g}}] \\
&= \mathbb{P}[\hat{\mu}_{a,\tilde{g},u_a} - \mu_{a,\tilde{g}} \geq \Delta_{a,\tilde{g}} \\
&\quad - \sigma_{\tilde{g}} \sqrt{\frac{2}{u_a} \log \frac{1}{\delta}}] \\
&\leq \mathbb{P}[\hat{\mu}_{a,\tilde{g},u_a} - \mu_{a,\tilde{g}} \geq c\Delta_{a,\tilde{g}}] \\
&\leq \exp\left(-\frac{u_a c^2 \Delta_{a,\tilde{g}}^2}{2\sigma_{\tilde{g}}^2}\right)
\end{aligned} \tag{4}$$

The last line of the equation 4 refers to a bounded error for a random variable following a Gaussian σ distribution. So $\mathbb{P}[G_a^C] \leq n\delta + \exp\left(-\frac{u_a c^2 \Delta_{a,\tilde{g}}^2}{2\sigma_{\tilde{g}}^2}\right)$. We looking for u_a that satisfy:

$$\begin{aligned}
\Delta_{a,\tilde{g}} - \sigma_{\tilde{g}} \sqrt{\frac{2}{u_a} \log \frac{1}{\delta}} &\geq c\Delta_{a,\tilde{g}} \\
u_a &\geq \frac{2\sigma_{\tilde{g}}^2 \log \frac{1}{\delta}}{\Delta_{a,\tilde{g}}^2 (1-c)^2}
\end{aligned}$$

If $\delta = \frac{1}{n^2}$

$$\begin{aligned}
\mathbb{E}[N_{a,\tilde{g}}(n)] &\leq \frac{2\sigma_g^2 \log \frac{1}{\delta}}{\Delta_{a,\tilde{g}}^2(1-c)^2} + 1 + n \exp\left(-\frac{u_a c^2 \Delta_{a,\tilde{g}}^2}{2\sigma_g^2}\right) \\
&\leq \frac{2\sigma_g^2 \log \frac{1}{\delta}}{\Delta_{a,\tilde{g}}^2(1-c)^2} + 1 + n^{-1+\frac{c^2}{(1+c)^2}}
\end{aligned} \tag{5}$$

The function $-1 + \frac{c^2}{(1+c)^2}$ is strictly negative and for any $c < \frac{1}{2}$. The equation 5 can be re written as :

$$\mathbb{E}[N_{a,\tilde{g}}(n)] \leq \frac{2\sigma_g^2 \log n^2}{\Delta_{a,\tilde{g}}^2 \frac{1}{2}} + 2$$

For any group $\tilde{g}$ the regret is bounded by:

$$\begin{aligned}
R_{n,\tilde{g}} &= \sum_{a \in \mathcal{A}} \Delta_{a,\tilde{g}} \mathbb{E}[N_{a,\tilde{g}}(n)] \\
&= \sum_{a \in \mathcal{A}} \frac{16\sigma_g^2 \log n^2}{\Delta_{a,\tilde{g}}} + 2 \sum_{a \in \mathcal{A}} \Delta_{a,\tilde{g}}
\end{aligned}$$

This finally define the cumulative regret for any group $\tilde{g}$ based on past data from $a = A$ (the original variation to improve). If the conditional inference tree define a set of $\mathcal{G}$ admissible groups where $\forall \tilde{g} \in \mathcal{G}$, $\mathcal{N}(\nu_{g,a=A}, \sigma_{g,a=A}^2)$ is statistically different (with an accepted risk of ϵ) and $|\mathcal{G}|$ the number of possible groups, the cumulative regret during the A/B-TEST is

$$\begin{aligned}
R_n^{\text{DBA-CTREE-UCB}} &= \sum_{\tilde{g} \in \mathcal{G}} R_{n,\tilde{g}} = \sum_{\tilde{g} \in \mathcal{G}} \left(\sum_{a \in \mathcal{A}} \frac{16\sigma_g^2 \log n^2}{\Delta_{a,\tilde{g}}} + 2 \sum_{a \in \mathcal{A}} \Delta_{a,\tilde{g}} \right) \\
&\leq |\mathcal{G}| \max_{\tilde{g} \in \mathcal{G}} [\sigma_g^2] \left(\sum_{a \in \mathcal{A}} \frac{16 \log n^2}{\Delta_{a,\tilde{g}}} + 2 \sum_{a \in \mathcal{A}} \Delta_{a,\tilde{g}} \right)
\end{aligned}$$